Fundamentals of Accelerated Data Science

NVIDIA Deep Learning Institute

What This Course Will Cover

An Introduction to Data Science with a Focus on Speed and Efficiency

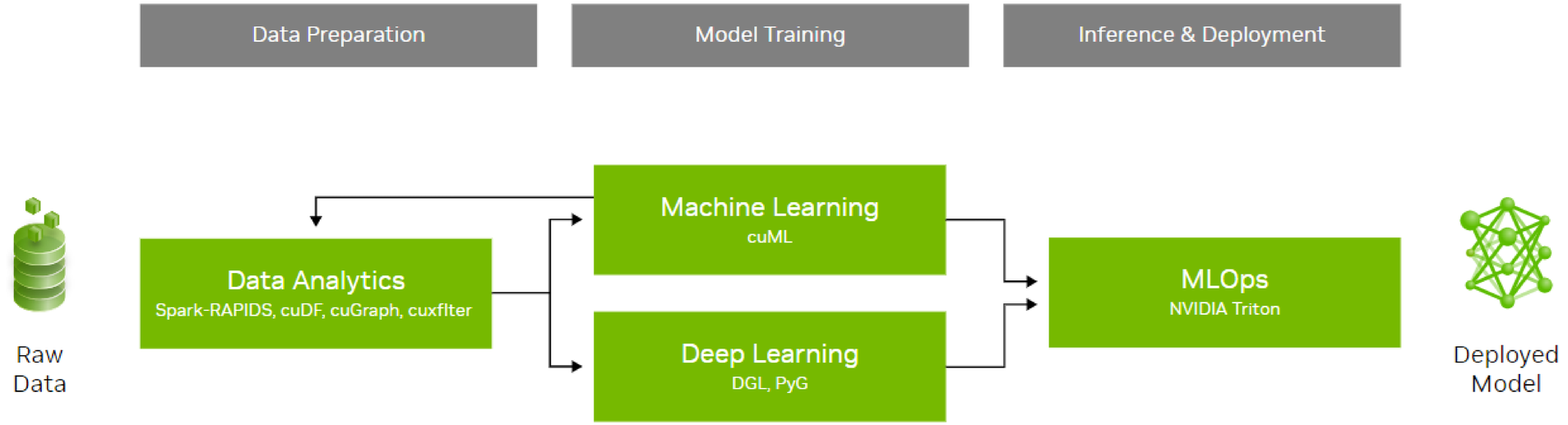
Learning objectives:

- Overview of data science
- Demonstrations of accelerated data science pipelines
- How acceleration is achieved and its implications

This workshop will not cover:

- Neural networks (accelerated by cuDNN)
- Statistical analysis

	Course Outline
Task 1	Data science overview Data manipulation
Task 2	Machine learning
Task 3	Graph analytics
Coding Assessment	Biodefense



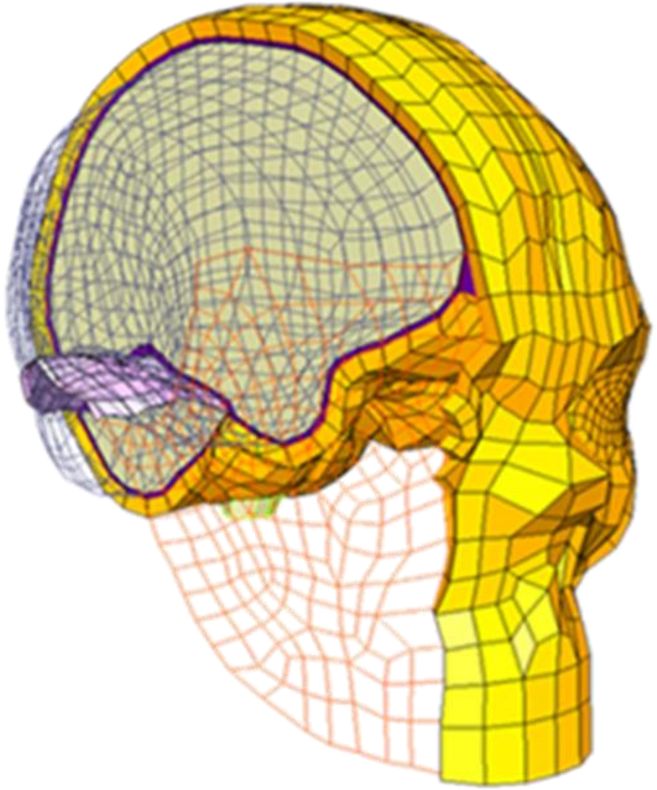
Agenda

- Graph 101
- Graph Analytics
- Hands-On Lab

Graph Data

Used for Representing Unstructured Data

VOLUMETRIC MESHES

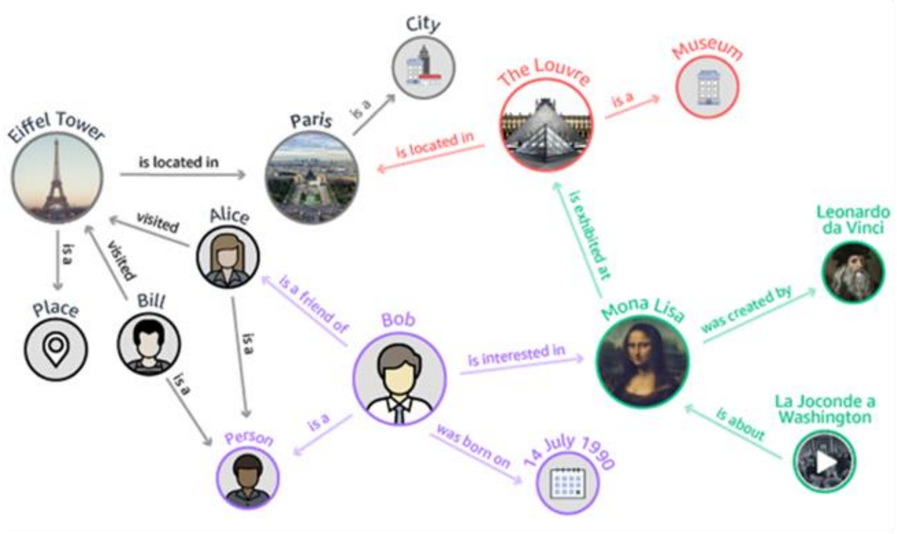


SURFACE MESHES

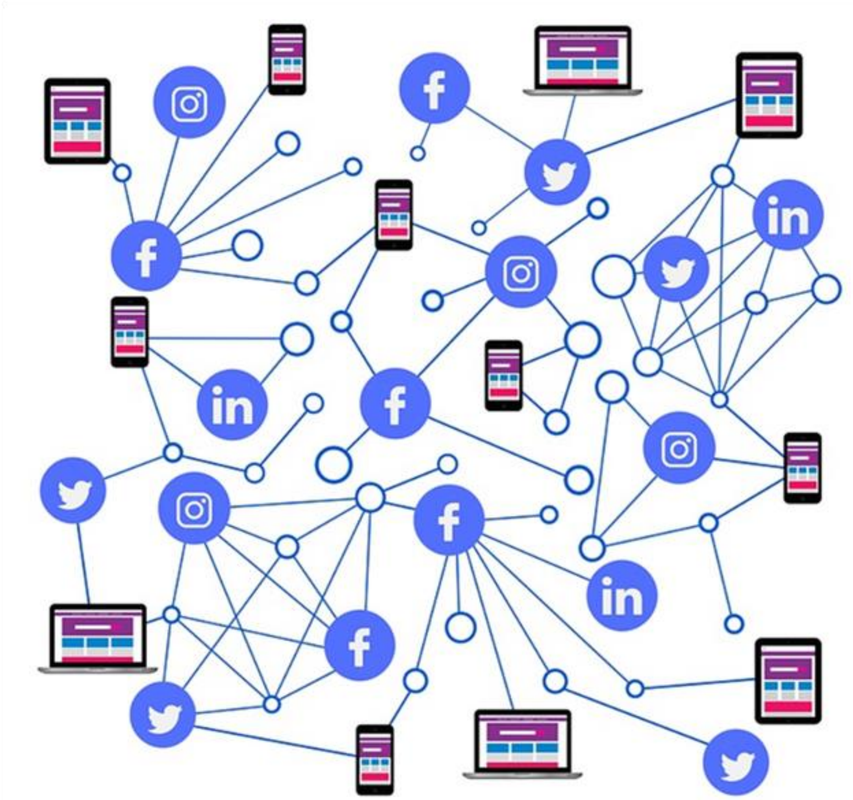
Common Graph Data Use Cases

Used to gather information about relationships between objects

KNOWLEDGE GRAPHS



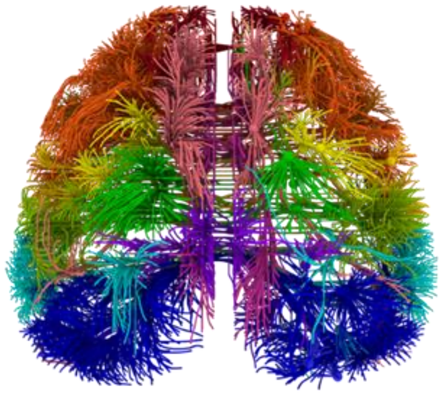
SOCIAL NETWORKS & RECOMMENDER SYSTEMS



Common Graph Data Use Cases

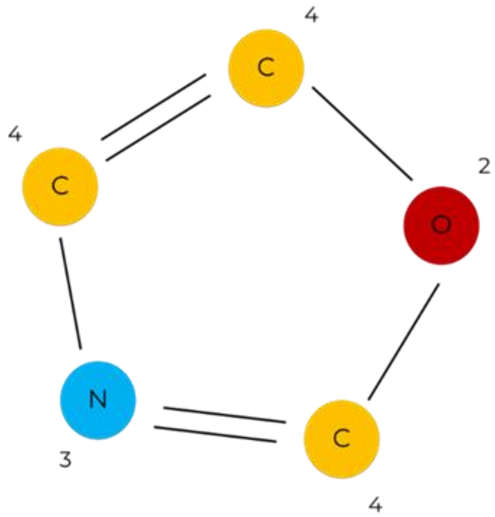
Usage extends across industries

Healthcare



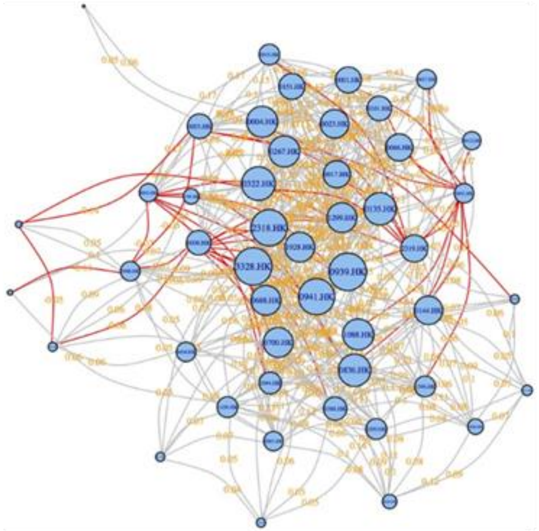
Connectomes
Brian fMRI/DTI

Chemistry & Pharmacy



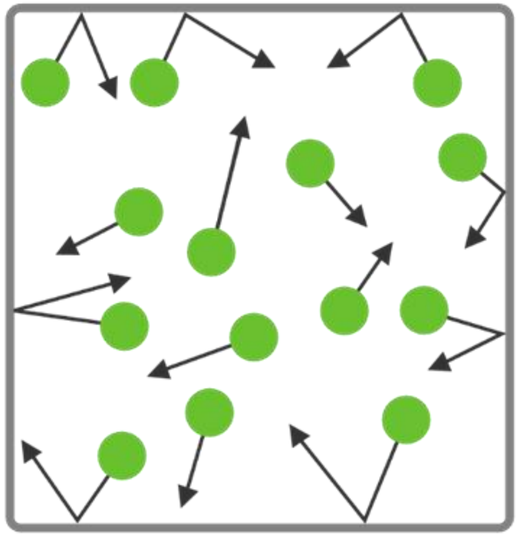
Molecular Analysis
Drug Discovery
Drug Repurposing

Financial Soundness Indicators



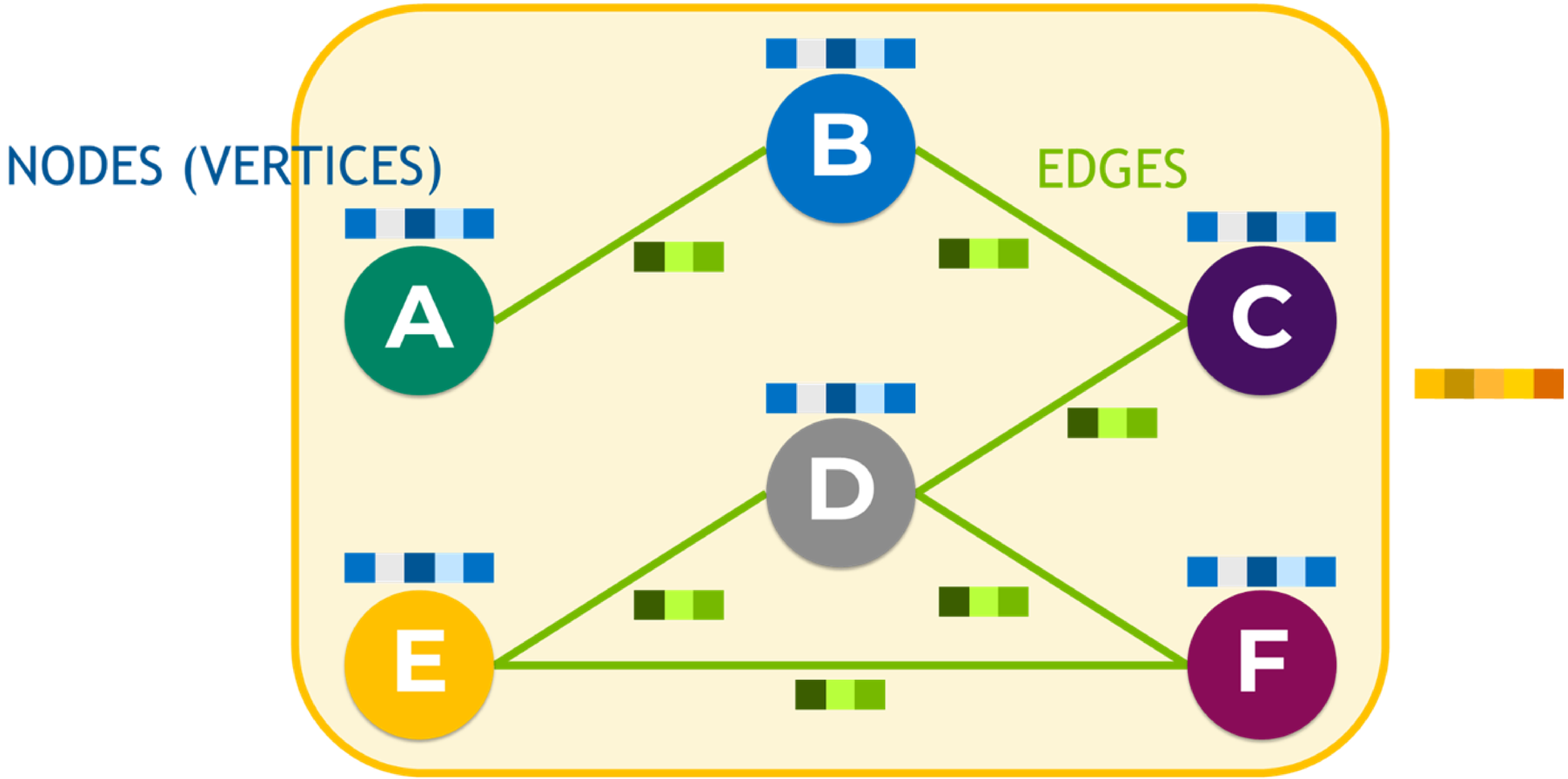
Stock Market Prediction

Physics



Particle Systems
Thermodynamics

Anatomy of a Graph

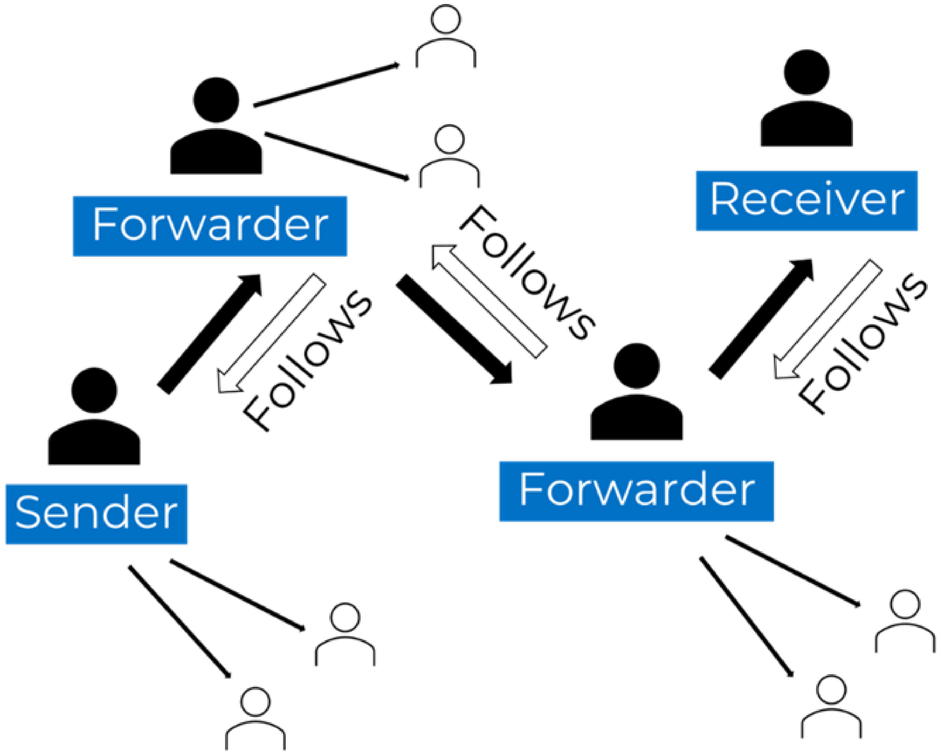


Edges

Directed, undirected, weighted, and unweighted

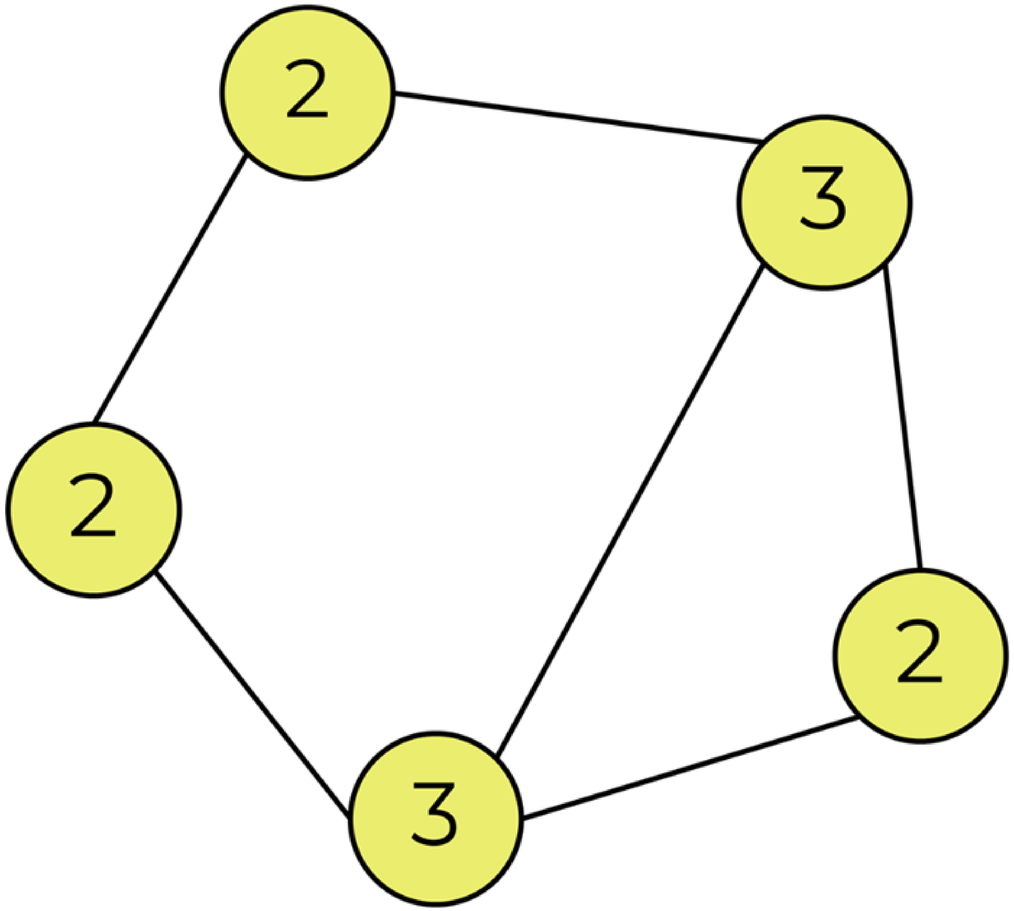
FACEBOOK

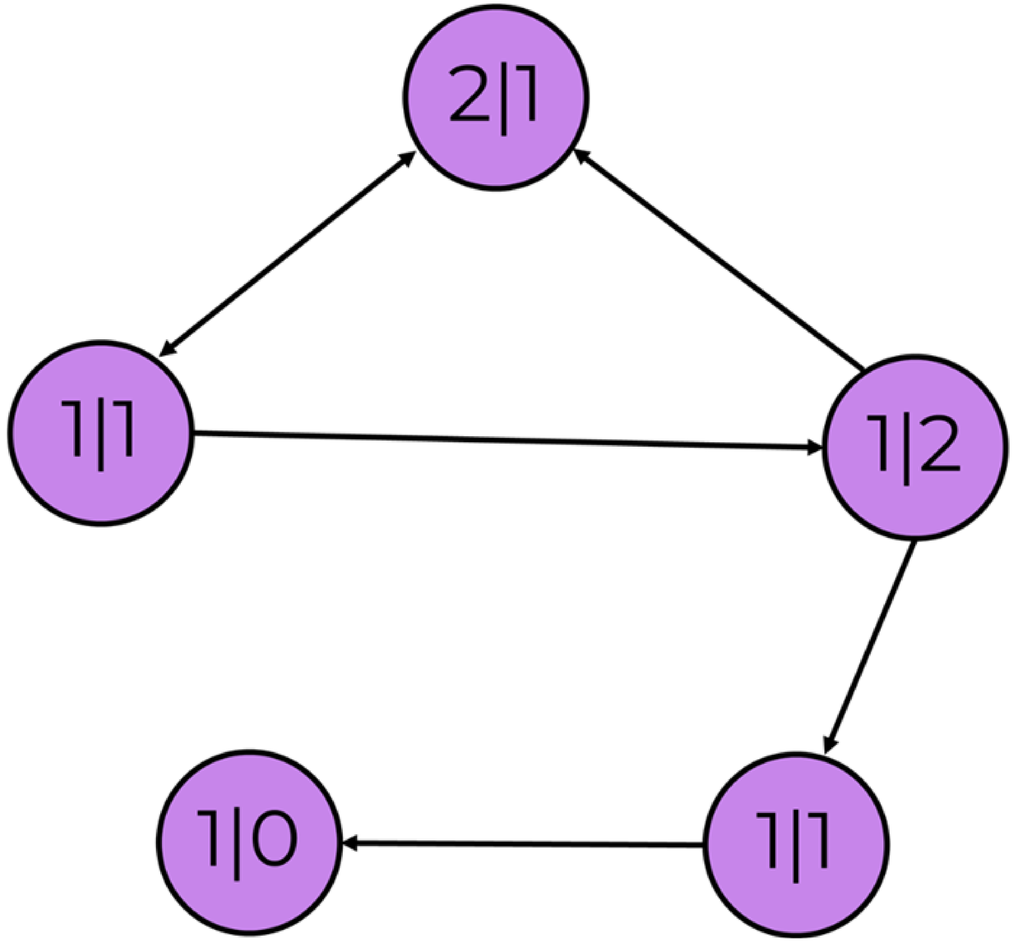
TWITTER



Edges

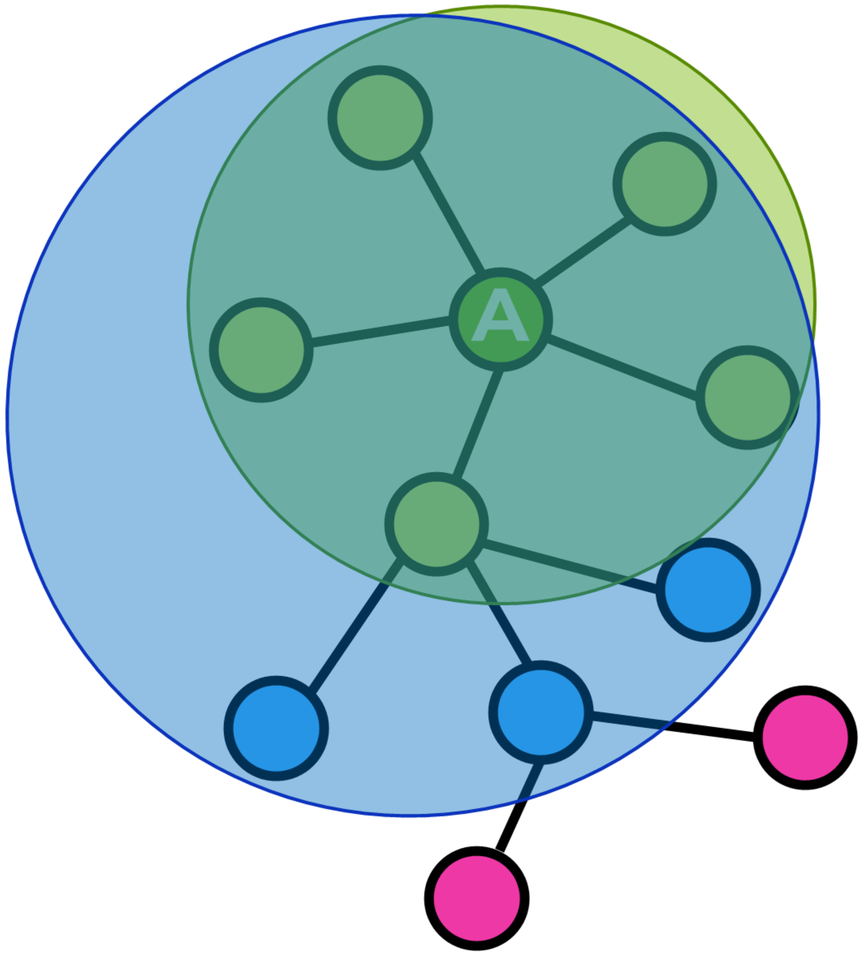
Degree, in-degree, out-degree





Edges

Neighborhood



1-HOP

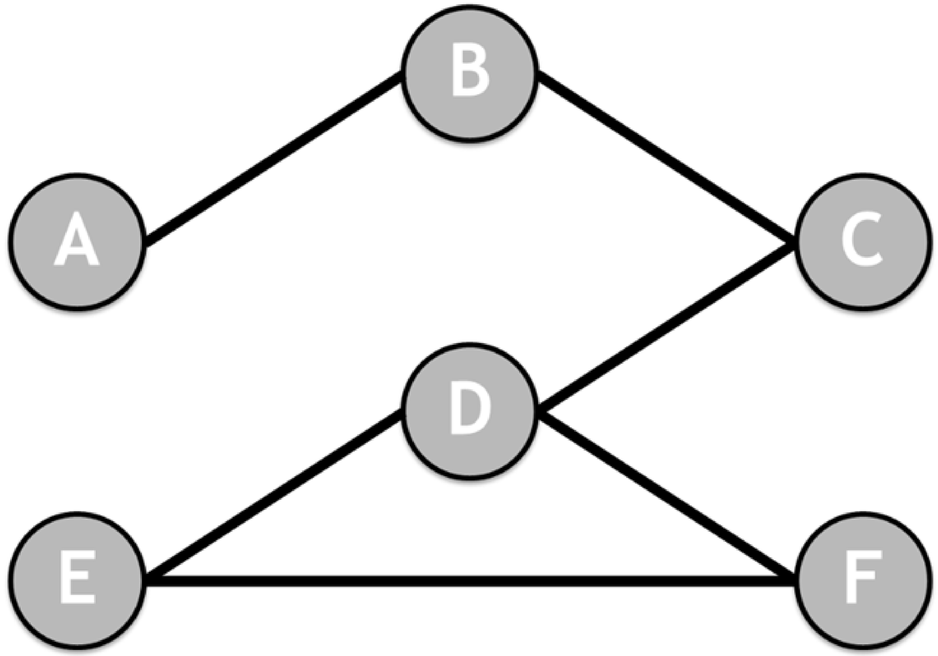
2-HOP

Representing Graphs

Adjacency matrix shows the neighborhood of each node

	A	B	C	D	E	F
A	0	1	0	0	0	0
B	1	0	1	0	0	0
C	0	1	0	1	0	0
D	0	0	1	0	1	1
E	0	0	0	1	0	1
F	0	0	0	1	1	0

Note: The adjacency matrix is symmetric for undirected graphs

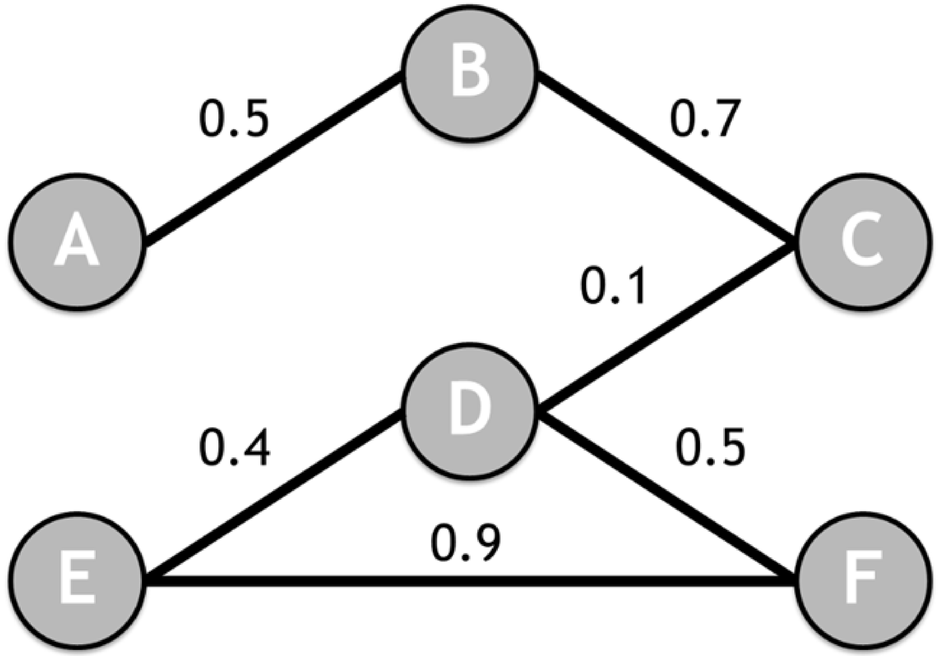


Representing Graphs

Adjacency matrix can also be used for a weighted graph

	A	B	C	D	E	F
A	0	0.5	0	0	0	0
B	0.5	0	0.7	0	0	0
C	0	0.7	0	0.1	0	0
D	0	0	0.1	0	0.4	0.5
E	0	0	0	0.4	0	0.9
F	0	0	0	0.5	0.9	0

Note: The adjacency matrix is symmetric for undirected graphs

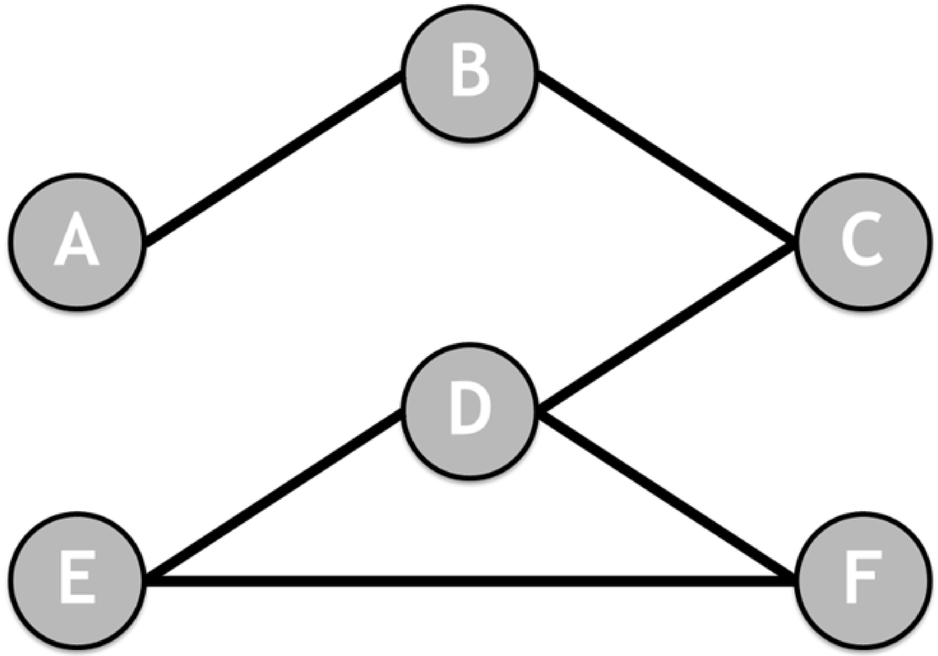


Representing Graphs

Degree matrix counts number of connections to each node

	A	B	C	D	E	F
A	2	0	0	0	0	0
B	0	2	0	0	0	0
C	0	0	2	0	0	0
D	0	0	0	3	0	0
E	0	0	0	0	2	0
F	0	0	0	0	0	2

Note: Illustration assumes unweighted and undirected graph



Representing Graphs

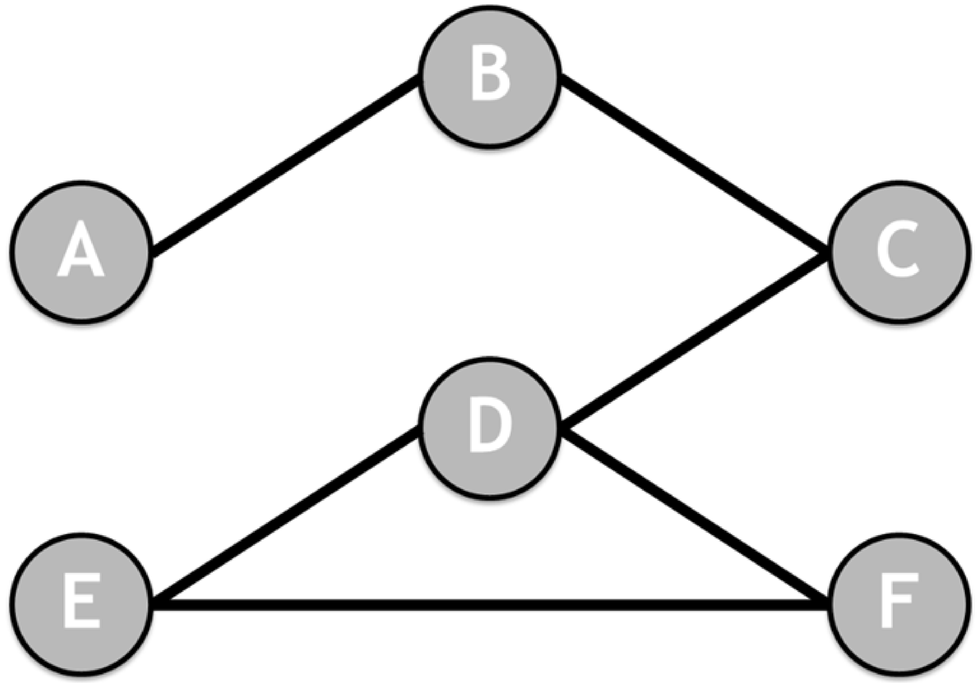
Laplacian matrix shows the smoothness of the graph

	A	B	C	D	E	F
A	2	-1	0	0	0	0
B	-1	2	-1	0	0	0
C	0	-1	2	-1	0	0
D	0	0	-1	3	-1	-1
E	0	0	0	-1	2	-1
F	0	0	0	-1	-1	2

Represented as L, where

L = { D - A }

D = Degree Matrix and
A = Adjacency Matrix

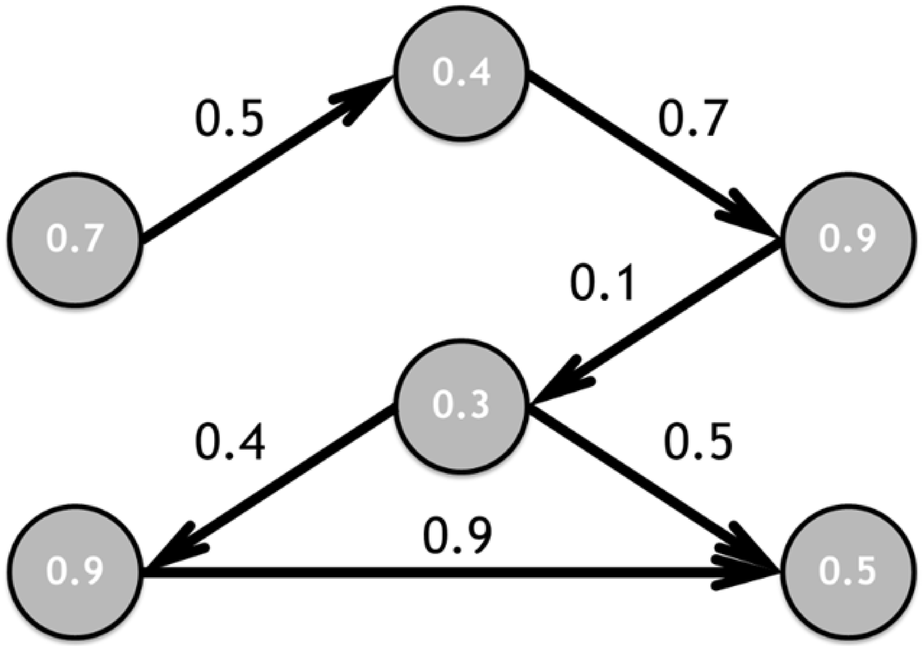


Representing Graphs

Graphs can also be represented by an adjacency list

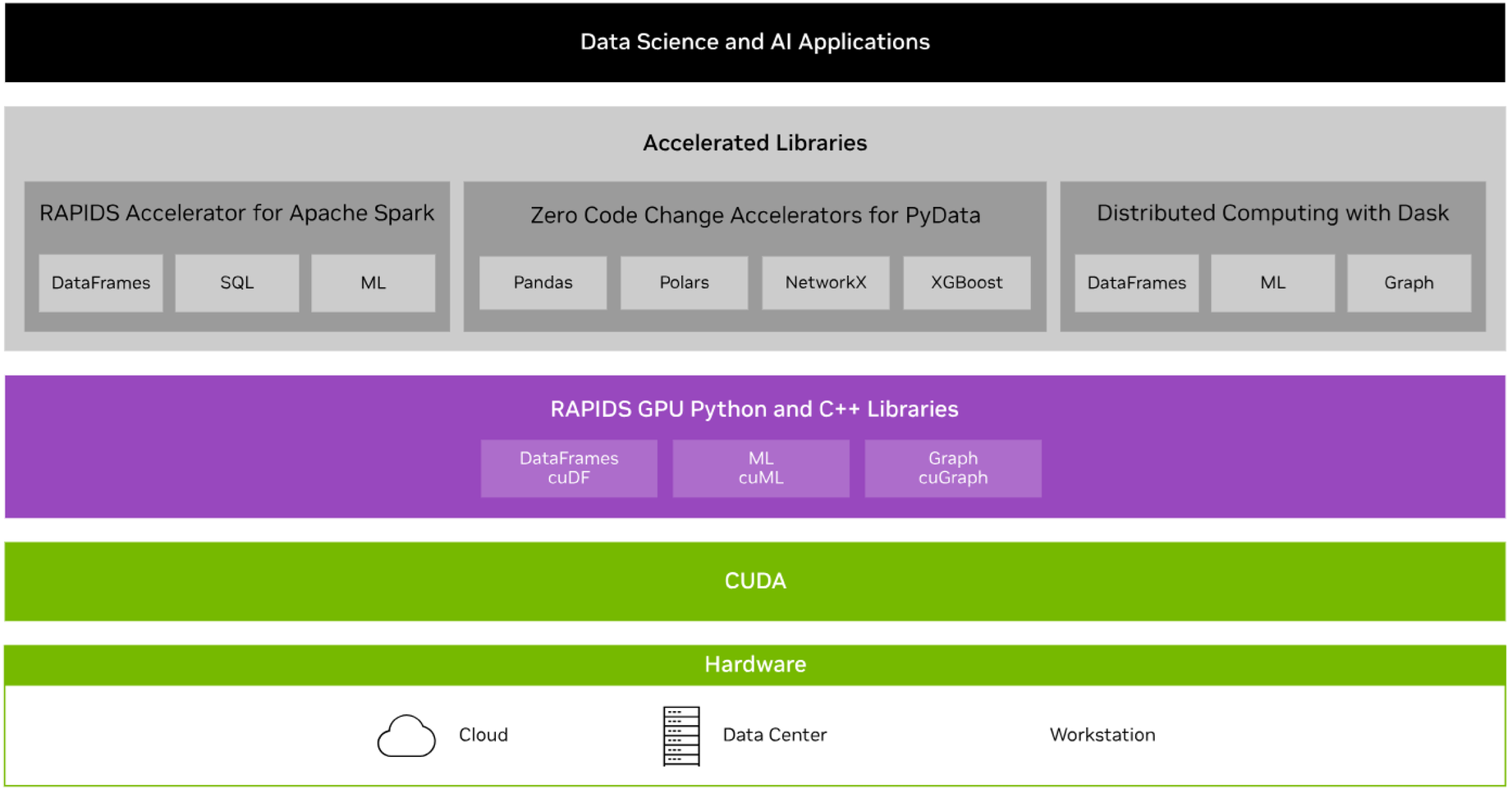
Adjacency	Edges	Nodes
[[1, 2], [2, 3], [3, 4], [4, 5], [4, 6], [5, 6]]	[0.5, 0.7, 0.1, 0.4, 0.5, 0.9]	[0.7, 0.4, 0.9, 0.3, 0.9, 0.5]

Note: Representing graph structure as adjacency lists can be more efficient



RAPIDS Transforms Data Science

An optimized hardware-to-software stack for the entire data science pipeline

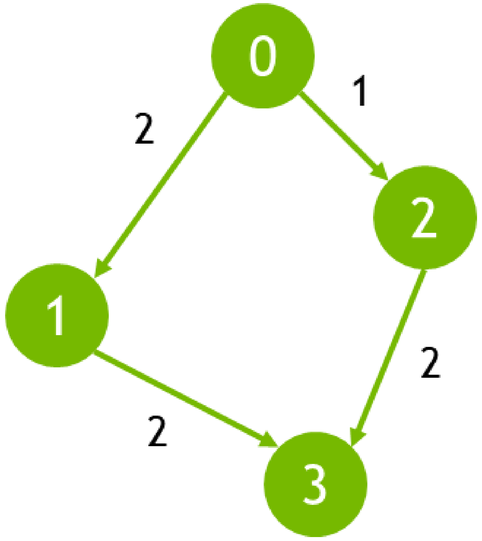


Single-Source Shortest Path

Computes the shortest path from a designated source node to all other reachable nodes in a weighted graph

Input: Graph (w/o negative-weight cycles), Node ID

Output: Vertex, Distance, Predecessor columns



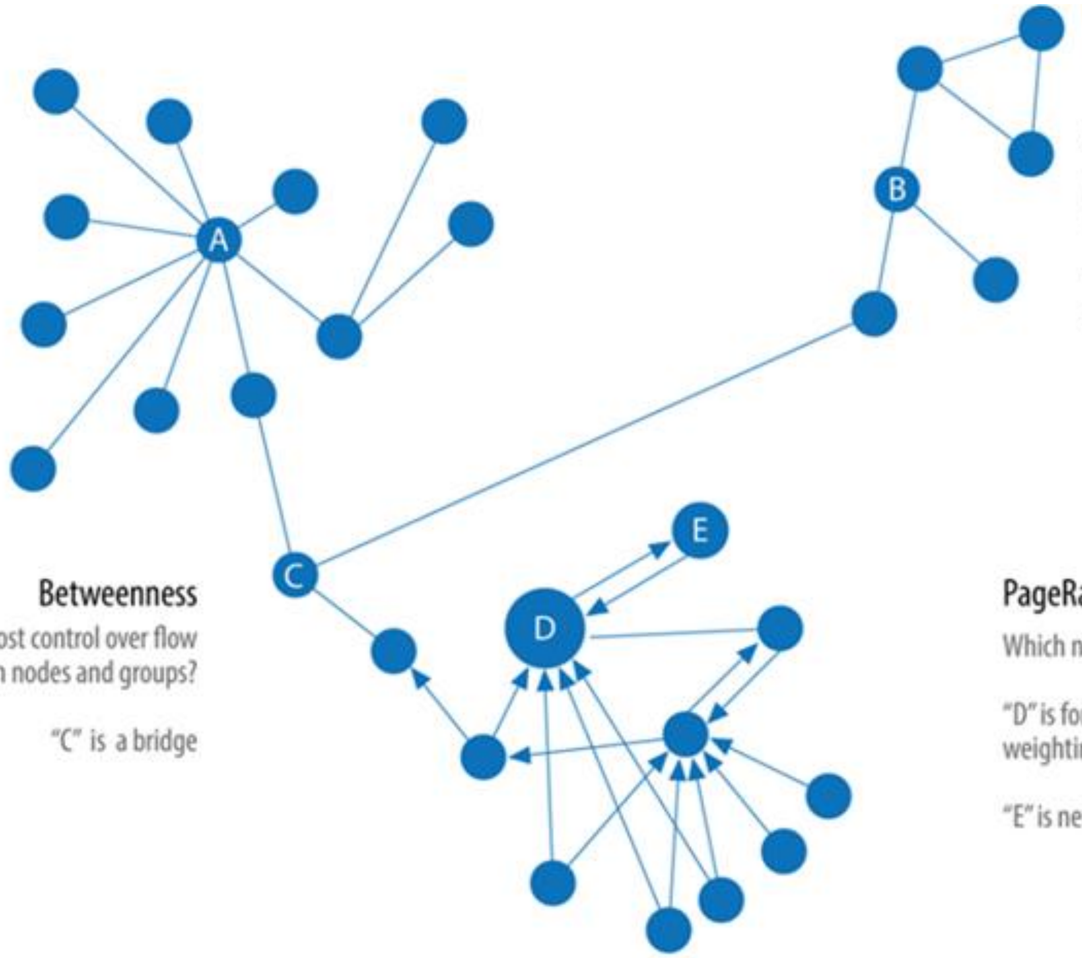
cg.sssp(G, 0)

Vertex	Distance	Predecessor
0	0	-1
1	2	0
2	1	0
3	3	2

Use cases: road networks, logistics, communications, and network analysis.

Centrality

Measuring different aspects of importance of nodes within a network



Degree

Number of connections?

"A" has a high degree

Closeness

Which node can most easily reach all other nodes in a graph or subgraph?

"B" is closest with the fewest hops in its subgraph

Betweenness

Which node has the most control over flow between nodes and groups?

"C" is a bridge

PageRank

Which node is the most important?

"D" is foremost based on number & weighting of in-links

"E" is next, due to the influence of D's link

nx-cugraph

NetworkX backend that provides GPU acceleration to many popular NetworkX algorithms

- Aims to bridge the gap between the ease of use of NetworkX and the high-performance capabilities of GPU-accelerated graph analytics

There are 3 ways to utilize nx-cugraph

1. Environment variable at runtime: the **NETWORKX_AUTOMATIC_BACKENDS** environment variable can be used to have NetworkX automatically dispatch to specified backends. This will use nx-cugraph to GPU-accelerate supported APIs with no code changes
2. Backend keyword argment: NetworkX also supports explicitly specifying a particular backend for supported APIs with the **backend** keyword argument
3. Type-Based dispatching: for users wanting to ensure a particular behavior, without the potential for runtime conversions, NetworkX offers type-based dispatching. To utilize this method, users must import the desired backend and create a Graph instance for it.

Graph Neural Networks

Learn the Function(s) to Predict Graph-Level, Node-Level, or Edge-Level Features